

Correction TD N°1

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

// Exercice 1

```
// Fonction pour afficher un tableau
```

```
void afficher_tableau(int t[], int taille) {
```

```
    int i;
```

```
    printf("Tableau : ");
```

```
    for (i = 0; i < taille; i++) {
```

```
        printf("%d ", t[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
// Fonction pour calculer la somme des éléments d'un tableau
```

```
int somme_tableau(int t[], int taille) {
```

```
    int i, somme = 0;
```

```
    for (i = 0; i < taille; i++) {
```

```
        somme += t[i];
```

```
    }
```

```
    return somme;
```

```
}
```

```
// Fonction pour rechercher une valeur dans le tableau
```

```
int recherche(int t[], int taille, int val) {
```

```
    int i;
```

```
    for (i = 0; i < taille; i++) {
```

```
        if (t[i] == val) {
```

```
            return i; // Retourne la position de l'élément trouvé
```

```
        }
```

```
    }
```

```
    return -1; // Retourne -1 si la valeur n'existe pas dans le tableau
```

```
}
```

```
// Fonction pour trier un tableau dans l'ordre croissant (tri par sélection)
```

```
void trier_tableau(int t[], int taille) {
```

```
    int i, j;
```

```
    for (i = 0; i < taille - 1; i++) {
```

```
        for (j = i + 1; j < taille; j++) {
```

```
            if (t[j] < t[i]) {
```

```
                int temp = t[i];
```

```
                t[i] = t[j];
```

```
                t[j] = temp; } }
```

```
}
```

Exercice 1 et 2 :

```
// Fonction pour afficher les nombres pairs du tableau
```

```
void nombre_pairs(int t[], int taille) {
```

```
    int i;
```

```
    printf("Nombres pairs dans le tableau : ");
```

```
    for (i = 0; i < taille; i++) {
```

```
        if (t[i] % 2 == 0) {
```

```
            printf("%d ", t[i]);
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
}
```

// Exercice 2

```
int max_tableau(int t[], int taille) {
```

```
    int max = t[0], i;
```

```
    for (i = 1; i < taille; i++) {
```

```
        if (t[i] > max) {
```

```
            max = t[i];
```

```
        }
```

```
    }
```

```
    return max;
```

```
}
```

```
void remplace_18(int t[], int taille) {
```

```
    int i;
```

```
    for (i = 0; i < taille; i++) {
```

```
        if (t[i] % 2 == 0) {
```

```
            t[i] = 18;
```

```
        }
```

```
    }
```

```
}
```

```
float moyenne_tableau(int t[], int taille) {
```

```
    int i, somme = 0;
```

```
    for (i = 0; i < taille; i++) {
```

```
        somme += t[i];
```

```
    }
```

```
    return (float)somme / taille;
```

```
}
```

// Main (Exercice 1 et 2)

```
int main() {  
  
    int i, taille t[50], val, pos;  
  
    printf("Entrez la taille du tableau : ");  
  
    scanf("%d", &taille);  
  
    // Saisie du tableau  
    printf("Entrez nombres entiers : ");  
  
    for (i = 0; i < taille; i++) {  
        scanf("%d", &t[i]);  
    }  
  
    afficher_tableau(t, taille);  
  
    // Calcul de la somme  
    printf("Somme des éléments : %d\n", somme_tableau(t, taille));  
  
    // Recherche d'une valeur  
    printf("Entrez une valeur à rechercher : ");  
  
    scanf("%d", &val);  
  
    pos = recherche(t, taille, val);  
  
    if (pos != -1) {  
        printf("Valeur trouvée à la position %d\n", pos);  
    } else {  
        printf("Valeur non trouvée\n");  
    }  
  
    // Tri et affichage du tableau trié  
    trier_tableau(t, taille);  
  
    printf("Tableau trié : ");  
  
    afficher_tableau(t, taille);  
  
    // Affichage des nombres pairs  
    nombre_pairs(t, taille);  
  
    printf("Le maximum du tableau est : %d\n", max_tableau(t, taille));  
  
    remplace_18(t, taille);  
  
    printf("Tableau après remplacement des nombres pairs par 18 : ");  
  
    afficher_tableau(t, taille);  
  
    printf("\nMoyenne du tableau : %.2f\n", moyenne_tableau(t, taille));  
  
    return 0;  
}
```

Exercice 1 et 2 :

```
#include <stdlib.h>

#include <stdio.h>

#define MAX_ETUDIANTS 100 // Nombre maximal d'étudiants

// Définition de la structure Etudiant

struct Etudiant {

    char nom[50];

    int age;

    float moyenne;

};

// Fonction pour saisir les informations d'un étudiant

void saisir_etudiants(struct Etudiant etudiants[], int n) {

    int i;

    for (i = 0; i < n; i++) {

        printf("\n=== Saisie de l'Etudiant %d ===\n", i + 1);

        printf("Nom : ");

        scanf("%s", etudiants[i].nom);

        printf("Age : ");

        scanf("%d", &etudiants[i].age);

        printf("Moyenne : ");

        scanf("%f", &etudiants[i].moyenne);

    }

}

// Fonction pour afficher les informations d'un étudiant

void afficher_etudiants(struct Etudiant etudiants[], int n) {

    int i;

    printf("\n===== Liste des etudiants =====\n");

    for (i = 0; i < n; i++) {

        printf("Etudiant %d :\n", i + 1);

        printf("Nom   : %s\n", etudiants[i].nom);

        printf("Age    : %d ans\n", etudiants[i].age);

        printf("Moyenne : %.2f\n", etudiants[i].moyenne);

        printf("-----\n");

    }

}
```

```
// Fonction pour afficher l'étudiant ayant la meilleure moyenne

void afficher_meilleur_etudiant(struct Etudiant etudiants[], int n) {

    int i, index_max;

    if (n == 0) {

        printf("Aucun etudiant enregistre.\n");

        return;

    }

    index_max = 0;

    for (i = 1; i < n; i++) {

        if (etudiants[i].moyenne > etudiants[index_max].moyenne) {

            index_max = i;

        }

    }

    printf("\n===== Meilleur Etudiant =====\n");

    printf("Nom    : %s\n", etudiants[index_max].nom);

    printf("Age    : %d ans\n", etudiants[index_max].age);

    printf("Moyenne : %.2f\n", etudiants[index_max].moyenne);

    printf("===== \n");

}

// Fonction principale

int main() {

    int n;

    struct Etudiant etudiants[MAX_ETUDIANTS];

    printf("Entrez le nombre d'etudiants : ");

    scanf("%d", &n);

    if (n <= 0 || n > MAX_ETUDIANTS) {

        printf("Nombre invalide !\n");

        return 1;

    }

    // Saisie des étudiants

    saisir_etudiants(etudiants, n);

    // Affichage des étudiants

    afficher_etudiants(etudiants, n);

    // Affichage du meilleur étudiant

    afficher_meilleur_etudiant(etudiants, n);

    return 0;

}
```

Exercice 4 :

```
#include <stdio.h>

#include <stdio.h>

#include <string.h>

int compter_voyelles(char str[]) {

    int i, count = 0;

    for (i = 0; str[i] != '\0'; i++) {

        char c = str[i];

        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||

            c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U') {

            count++;

        }

    }

    return count;

}

// Fonction pour inverser une chaîne

void inverse_chaine(char str[]) {

    int i, n = strlen(str);

    printf("Chaîne inversée : ");

    for (i = n - 1; i >= 0; i--) {

        printf("%c", str[i]);

    }

    printf("\n");

}

// Fonction pour vérifier si une chaîne est un palindrome

int est_palindrome(char str[]) {

    int i, n = strlen(str);

    for (i = 0; i < n / 2; i++) {

        if (str[i] != str[n - i - 1]) {

            return 0; // Ce n'est pas un palindrome

        }

    }

    return 1; // C'est un palindrome

}
```

```
int main() {

    char chaine[100];

    printf("Entrez une chaîne de caractères : ");

    scanf("%s", chaine);

    printf("Longueur de la chaîne : %d\n", strlen(chaine));

    printf("Nombre de voyelles : %d\n", compter_voyelles(chaine));

    inverse_chaine(chaine);

    if (est_palindrome(chaine)) {

        printf("La chaîne est un palindrome.\n");

    } else {

        printf("La chaîne n'est pas un palindrome.\n");

    }

    return 0;

}
```

Exercice 5 :

```
#include <stdio.h>

#include <stdlib.h>

// Fonction pour calculer la fréquence d'une valeur dans le tableau
int frequence(int *tableau, int taille, int val) {

    int i, count = 0;

    for (i = 0; i < taille; i++) {

        if (tableau[i] == val) {

            count++;

        }

    }

    return count;

}

// Fonction pour afficher les nombres se terminant par 8
void affiche_8(int *tableau, int taille) {

    int i;

    printf("Nombres se terminant par 8 : ");

    for (i = 0; i < taille; i++) {

        if (tableau[i] % 10 == 8) {

            printf("%d ", tableau[i]);

        }

    }

    printf("\n");

}

// Fonction pour calculer et afficher les multiples de 3
void nombre_3(int *tableau, int taille) {

    int i, count = 0;

    printf("Multiples de 3 : ");

    for (i = 0; i < taille; i++) {

        if (tableau[i] % 3 == 0) {

            printf("%d ", tableau[i]);

            count++;

        }

    }

    printf("\nNombre total de multiples de 3 : %d\n", count);

}
```

```
int main() {

    int i, *tableau, taille, val, freq;

    // Demander à l'utilisateur la taille du tableau

    printf("Entrez la taille du tableau : ");

    scanf("%d", &taille);

    // Allouer dynamiquement la mémoire pour le tableau

    tableau = (int *)malloc(taille * sizeof(int));

    if (tableau == NULL) {

        printf("Erreur d'allocation de mémoire\n");

        return 1;

    }

    // Permettre à l'utilisateur d'entrer les valeurs

    printf("Entrez les valeurs du tableau :\n");

    for (i = 0; i < taille; i++) {

        printf("Element %d : ", i + 1);

        scanf("%d", &tableau[i]);

    }

    // Afficher les valeurs du tableau

    printf("Valeurs du tableau : ");

    for (i = 0; i < taille; i++) {

        printf("%d ", tableau[i]);

    }

    printf("\n");

    // Demander à l'utilisateur une valeur pour calculer sa fréquence

    printf("Entrez une valeur pour calculer sa fréquence : ");

    scanf("%d", &val);

    freq = frequence(tableau, taille, val);

    printf("La fréquence de %d dans le tableau est : %d\n", val, freq);

    // Afficher les nombres se terminant par 8

    affiche_8(tableau, taille);

    // Calculer et afficher les multiples de 3

    nombre_3(tableau, taille);

    // Libérer la mémoire allouée

    free(tableau);

    return 0;

}
```